

# Computation and Boundary Evidence for Scientific Software Repair

Rajarshi Ghoshal

rajarshi.ghoshal1@gmail.com

PhD applicant research task — Graz University of Technology

## Abstract

A repair agent with shell tools must reconstruct a repository’s computational structure from text search. CBE (computation and boundary evidence) prepares that structure once and lets the agent query it during repair, under the same model and budget. On 89 held-out SWE-bench Science tasks ( $k=3$ ), both arms solve 60 attempts (22.9% vs. 23.1%). Aggregate parity masks a discipline-level split: mechanics +22.2 pp, materials +9.7 pp, astronomy +12.4 pp; biomedical engineering −16.7 pp, biology −9.5 pp. CBE generates 13.7% fewer output tokens. About a fifth of tasks flip between identical runs, bounding what any single-run comparison can resolve.

## 1 Introduction

Repairing a composite-laminate stiffness calculation requires changing both the ply iteration and the accumulation formula. Expanding one without the other passes the public check but fails the private tests. The patch is short; finding it means knowing which expression produces the returned matrix and what calls cross the repository boundary.

We evaluate whether providing this structure directly changes repair outcomes. CBE exposes source-linked computation structure; the baseline reconstructs the same context from `grep` and file reads. Same model, same budget.

Contributions: the preparation pipeline, the query interface, and a replicated evaluation that separates discipline-level differences from aggregate noise.

CBE draws on traceability (Niu et al., 2023b,a, 2025; Pan et al., 2025), code property graphs (Yamaguchi et al., 2014), scientific model extraction (Pyarelal et al., 2020; ML4AI Team, 2024), and incremental parsing (Brunsfield et al., 2024). We evaluate on SWE-bench Science (Xu et al., 2026) with DeepSeek V4.1 Flash (DeepSeek, 2026); SWE-bench (Jimenez et al., 2024; Yang et al., 2024) and SciCode (Tian et al., 2024) are related benchmarks.

## 2 What CBE provides

**Prepare.** Run the public `reproduce.py` under an observer. Observed functions guide file selection. Static extraction adds expressions, conditions, definitions, and call boundaries without model calls. Median duration: 73 s of the 1800 s budget.

**Table 1.** CBE evidence records for task 103 (pyNastran).

| Record      | Content                                                                                       |
|-------------|-----------------------------------------------------------------------------------------------|
| Computation | Scope, expressions, result anchors: accumulation at line 1003 → return ABD at 1015.           |
| Condition   | <code>len(z0) == len(z1)</code> at line 987.                                                  |
| Definition  | Documented public behavior (not execution trace).                                             |
| Boundary    | <code>np.radians(thetad)</code> at line 979: provider <code>numpy</code> , caller-side scope. |

### Query interface (alongside ordinary shell)

|                      |                                                           |
|----------------------|-----------------------------------------------------------|
| <code>find</code>    | Keyword search over symbols, docstrings, and expressions. |
| <code>inspect</code> | Structured view: dependencies, conditions, boundaries.    |
| <code>note</code>    | Scratchpad for working hypotheses; never gates repair.    |

**Represent.** Tree-sitter (Brunsfield et al., 2024) parses Python ASTs for scopes, assignments, binary expressions, return anchors, and branches. Jern handles C, C++, Fortran, and Cython where available; explicit fallback otherwise. External calls are labelled with their provider as interface assumptions, not correctness guarantees. Unresolved callees stay `unknown_external`.

**Query and repair.** The agent accesses three endpoints alongside shell utilities: `find` searches extracted symbols, expressions, and docstrings; `inspect` retrieves computation scopes, dependencies, and boundary assumptions; `note` records an explicit working model into a durable scratchpad without gating repair actions. Inspection dynamically refreshes edited source lines, while preparation traces remain fixed.

### 3 Protocol

119 tasks, 98 repositories, 20 disciplines (Xu et al., 2026). 30 tasks held for development, 89 locked for evaluation.

**Table 2.** Shared evaluation parameters across both arms.

| Component    | Detail                                                                         |
|--------------|--------------------------------------------------------------------------------|
| Model        | DeepSeek V4.1 Flash (DeepSeek, 2026); temperature 0; PYTHONHASHSEED=0.         |
| Replicates   | 89 tasks $\times$ 3 runs = 267 scheduled per arm; 522 verified.                |
| Budget       | 1800 s, including preparation.                                                 |
| Verification | Separate container, 1800 s, runs after repair ends. Private tests not visible. |
| Network      | None.                                                                          |
| Baseline     | Shell tools + public task material.                                            |
| CBE          | Same + evidence store + three query endpoints.                                 |

A solve is the official verifier reward. Missing verdicts excluded, not failures. Paired test: bootstrap per-task rate differences (20k resamples, fixed seed). Sign test excludes ties.

Each trial executes in an isolated Docker container with 8 CPUs and 16 GiB RAM under a strict 1800 s total budget. Evaluation applies the final git patch in a clean container and executes the unobserved private test suite.

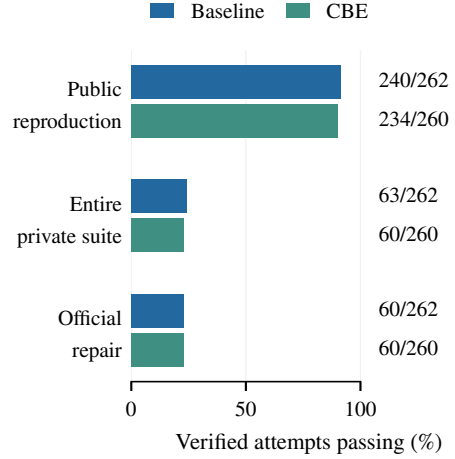
### 4 Parity, fewer tokens

**Table 3.** Both arms solve 60. CBE generates 13.7% fewer output tokens.

| Metric                    | Baseline  | CBE       |
|---------------------------|-----------|-----------|
| Scheduled attempts        | 267       | 267       |
| Exact repairs / verdicts  | 60/262    | 60/260    |
| Exact-repair rate         | 22.9%     | 23.1%     |
| Private tests passed      | 2315/3167 | 2226/3148 |
| Output tokens (M)         | 12.01     | 10.36     |
| Input tokens (M)          | 838.0     | 878.3     |
| Mean work per attempt (s) | 1236.6    | 1306.9    |

Paired task-rate diff: +0.75 pp; 95% bootstrap  $[-3.4, +4.9]$  pp; sign  $p = 1.00$ .

74 tasks tie, 7 favour CBE, 8 favour baseline. Output drops 13.7% (10.36M vs. 12.01M tokens). Input rises 4.8%. Work takes 70 s longer. The agent searches less — it either finds the right computation or anchors on the wrong one.



**Figure 1.** Public reproducer vs. private verifier pass rates. Most attempts pass the public check without satisfying the full verifier.

CBE avoids speculative file reading and iterative grep pipelines. By querying pre-indexed computation graphs and boundary signatures directly, the agent issues fewer exploratory turns, reducing total output generation across verified attempts.

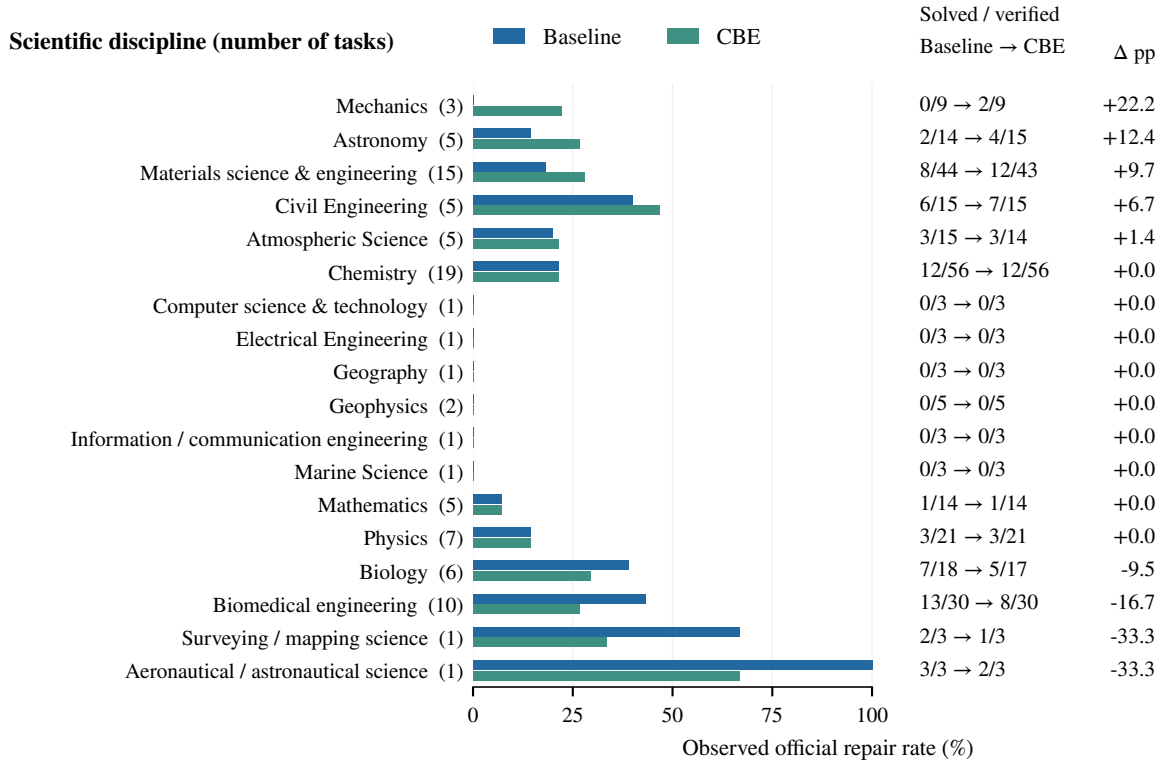
**Hidden partial changes.** Of 74 exact ties, 30 differ in private-test coverage (Table 4).

**Table 4.** Exact-rate ties crossed with private-test coverage.

| Exact rate      | Private-test coverage |       |            |
|-----------------|-----------------------|-------|------------|
|                 | Baseline higher       | Equal | CBE higher |
| Baseline higher | 7                     | 0     | 1          |
| Equal           | 21                    | 44    | 9          |
| CBE higher      | 0                     | 0     | 7          |

Of the 74 exact ties, 30 show divergence on held-out private tests: CBE improves test coverage in 9 tasks without changing binary solve status, while baseline achieves higher coverage on 21. For tasks that both arms solve, coverage is identical in 44 instances.

## 5 Discipline split



**Figure 2.** Official solve rate by scientific discipline across all 18 represented domains (task counts in parentheses; right column is percentage-point difference  $\Delta$ ).

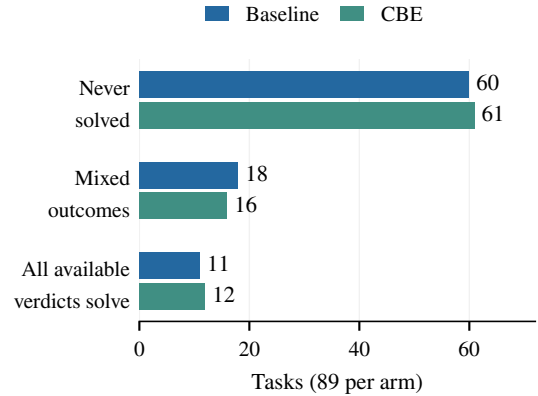
Mechanics: +22.2 pp, entirely task 103 (pyNas-tran). The agent inspects the ply-angle computation and its numpy boundaries, then edits — 2/3 solves vs. 0 baseline. Materials: +9.7 pp across three tasks. Astronomy: +12.4 pp.

Biomedical engineering: −16.7 pp across four tasks. Task 022 (nilearn): baseline solves 2/3, CBE 0/3. The trajectory reads projection mathematics before touching the atlas-label mapping. Biology: −9.5 pp. The two −33.3 pp domains are single-tons.

Chemistry (19 tasks), physics (7) and mathematics (5) do not move. No interaction with the ablation flag: −0.3 pp on ablated tasks, +1.4 pp on knowledge-provided ones. 81 of 89 tasks are pure Python, so language strata are uninformative.

When the defect is a computation, structure finds it. When an ordinary bug sits under scientific prose, structure distracts.

**Replication stability.** Most tasks never solve or always solve across repeated runs; about a fifth flip within an arm (Figure 3).



**Figure 3.** Within-arm outcome stability across 3 repeated seeds. Most tasks are deterministically solved or unsolved; 18–20% flip between runs.

Across the 89 tasks, 60 never solve in either arm, 11–12 solve in all three runs, and 16–18 yield mixed outcomes. This within-arm stochasticity bounds single-run benchmark resolution: differences smaller than  $\sim 5$  pp fall within stochastic run-to-run noise.

## 6 Case studies

| Task / repository    | Discipline   | Baseline     |              |              | CBE          |              |              | Solved / verified |
|----------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|-------------------|
|                      |              | Run 1        | Run 2        | Run 3        | Run 1        | Run 2        | Run 3        |                   |
| 074 DeePTB           | Materials    | 9/10         | 9/10         | 2/10         | <b>10/10</b> | <b>10/10</b> | —            | 0/3 → 2/2         |
| 042 sunpy            | Astronomy    | 15/16        | 15/16        | 6/16         | <b>16/16</b> | 7/16         | <b>16/16</b> | 0/3 → 2/3         |
| <b>103 pyNastran</b> | Mechanics    | 2/4          | 2/4          | 2/4          | <b>4/4</b>   | <b>4/4</b>   | 2/4          | 0/3 → 2/3         |
| 067 xclim            | Atmospheric  | <b>6/6</b>   | <b>6/6</b>   | 3/6          | <b>6/6</b>   | <b>6/6</b>   | —            | 2/3 → 2/2         |
| 020 deepmd-kit       | Materials    | 10/12        | 10/12        | 11/12        | 10/12        | <b>12/12</b> | 10/12        | 0/3 → 1/3         |
| 084 dpdata           | Materials    | 5/7          | 5/7          | <b>7/7</b>   | <b>7/7</b>   | 5/7          | <b>7/7</b>   | 1/3 → 2/3         |
| 116 Pynite           | Civil eng.   | 85/93        | 85/93        | 85/93        | 85/93        | <b>93/93</b> | 85/93        | 0/3 → 1/3         |
| 029 biopython        | Biology      | <b>29/29</b> | 14/29        | 27/29        | 14/29        | 14/29        | 14/29        | 1/3 → 0/3         |
| 053 MONAI            | Biomedical   | 2/6          | <b>6/6</b>   | 2/6          | 5/6          | 5/6          | 2/6          | 1/3 → 0/3         |
| 063 tskit            | Biology      | 9/18         | <b>18/18</b> | 7/18         | 7/18         | 9/18         | —            | 1/3 → 0/2         |
| 087 dipy             | Biomedical   | <b>5/5</b>   | 0/5          | 1/5          | 0/5          | 2/5          | 2/5          | 1/3 → 0/3         |
| 093 MONAI            | Biomedical   | <b>4/4</b>   | 2/4          | 2/4          | 2/4          | 2/4          | 2/4          | 1/3 → 0/3         |
| 098 satpy            | Surveying    | <b>11/11</b> | 8/11         | <b>11/11</b> | 8/11         | <b>11/11</b> | 8/11         | 2/3 → 1/3         |
| 056 poliaastro       | Aeronautical | <b>10/10</b> | <b>10/10</b> | <b>10/10</b> | <b>10/10</b> | <b>10/10</b> | 1/10         | 3/3 → 2/3         |
| <b>022 nilearn</b>   | Biomedical   | <b>12/12</b> | <b>12/12</b> | 10/12        | 4/12         | 11/12        | 11/12        | 2/3 → 0/3         |

Cell: private tests passed / collected. Outlined + bold: official repair. Dash: no verdict.

**Figure 4.** Per-run private-test fractions for every task whose solve rate differs between arms. Outlined bold cells are official solves; bold task labels are the two cases below.

**Table 5.** Comparison of generated patches for divergent case studies.

| Task             | Baseline patches                                                                       | CBE patches                                                                                             | What it shows                                                                                                                                                            |
|------------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 103<br>pyNastran | Expand ply iteration, keep original B/D accumulation. All three pass the public check. | Two change both material-reference expansion and B/D expressions; the third keeps the old accumulation. | Public checks undertest multi-step calculations. Exposing complete computation structure separates full repairs from partial workarounds.                                |
| 022<br>nilearn   | Add label-aggregation option without altering the existing nearest-neighbour path.     | Add label-aware aggregation but alter argument handling in the existing path; 11/12 private tests pass. | Adding a capability while preserving the existing interface is the repair obligation. Focusing on internal mathematics caused the agent to alter the interface contract. |

Task 103: covering the full computation, not just the example, is the repair obligation. Task 022: preserving the existing interface while adding the new case. CBE helped with the first and hurt the second.

**Data and reproduction.** Full trajectories, unified git diffs, agent transcripts, and verifier receipts are archived in the repository. The evaluation environment is fully deterministic under pinned container images and fixed random seeds.

## 7 Limitations and conclusion

One model, one effort, one budget. Domain analyses are descriptive; singleton domains carry no weight. The store is approximate: unparseable expressions are recorded as unsupported, external calls are labelled rather than verified. No ablations isolate which part of CBE matters.

CBE reaches parity, generates fewer tokens, and splits the benchmark by discipline. Scientific structure helps when the defect is a computation and hurts when it is ordinary code.

**Artifacts and AI use.** The author designed the study; LLM assistants (Oh My Pi, ChatGPT) helped implement the harness and revise prose. DeepSeek V4.1 Flash (DeepSeek, 2026) was the only repair model.

## References

- Max Brunsfeld et al. Tree-sitter: An incremental parsing system for programming tools, 2024. URL <https://tree-sitter.github.io>.
- DeepSeek. DeepSeek V4.1 Flash: Model card and API documentation, 2026. URL <https://api-docs.deepseek.com>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.
- ML4AI Team. GroMEt: A unified intermediate representation for scientific models and code. *DARPA ASKEM / SKEMA Consortium*, 2024.
- Feifei Niu, Wesley K. G. Assunção, Liguó Huang, Christoph Mayr-Dorn, Jidong Ge, Bin Luo, and Alexander Egyed. RAT: A refactoring-aware traceability model for bug localization. In *Proc. 45th International Conference on Software Engineering (ICSE)*, 2023a.
- Feifei Niu, Christoph Mayr-Dorn, Wesley K. G. Assunção, Liguó Huang, Jidong Ge, Bin Luo, and Alexander Egyed. The ABLoTS approach for bug localization: Is it replicable and generalizable? In *Proc. 20th International Conference on Mining Software Repositories (MSR)*, pages 152–162, 2023b.
- Feifei Niu, Liguó Huang, Christoph Mayr-Dorn, and Alexander Egyed. TVR: Automotive system requirement traceability validation and recovery through retrieval-augmented generation. *arXiv preprint arXiv:2504.15427*, 2025.
- Xinyi Pan, Feifei Niu, Liguó Huang, and Alexander Egyed. Requirements coverage-guided minimization for natural language test cases. *arXiv preprint arXiv:2505.20004*, 2025.
- Adarsh Pyarelal et al. AutoMATES: Automated model assembly from text, equations, and software. *arXiv preprint arXiv:2001.07295*, 2020.
- Minyang Tian, Luyu Gao, Shizhuo Dylan Zhang, et al. SciCode: A research coding benchmark curated by scientists. *arXiv preprint arXiv:2407.13168*, 2024.
- Zhipeng Xu, Jiahao Lu, Yining Zheng, Yuxin Wang, and Xipeng Qiu. SWE-bench Science: Can coding agents resolve engineering tasks in science? *arXiv preprint arXiv:2608.19799v2*, 2026.
- Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. Modeling and discovering vulnerabilities with code property graphs. In *IEEE Symposium on Security and Privacy*, 2014. doi: 10.1109/SP.2014.44.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-bench Verified: A stricter subset for real-world software engineering. *arXiv preprint arXiv:2407.01489*, 2024.